Evaluación Práctica de la Unidad 2

Inferencia Estadística y Toma de Decisiones en Ingeniería

Caso A: Arquitectura de Software (Monolito vs. Microservicios)

Hernandez Barreto Alan Daleth
Alfonso Dominguez Gonzales

TecNM – Departamento de Sistemas y Computación
Ingeniería en Sistemas Computacionales
Asignatura: Fundamentos de Ciencia de Datos

Octubre 2026

Resumen

Se evalúa si la sobrecarga de red de una arquitectura de microservicios incrementa la latencia de respuesta HTTP respecto a un monolito. A partir de un dataset sintético de 500 observaciones (250 por grupo) depurado con el criterio 1,5×IQR (483 registros), se aplicaron estadística descriptiva, simulación del Teorema Central del Límite (TCL), intervalos de confianza y una prueba t de Student bilateral. El IC al 95 % para la diferencia de medias fue $[-9{,}63,\ 6{,}80]$ ms y la prueba arrojó $t = -0{,}34$ con $p = 0{,}74$: no se rechaza la hipótesis nula. La latencia media no difiere significativamente entre arquitecturas en este experimento.

1. Introducción y objetivo

En sistemas distribuidos se presume que la comunicación entre servicios agrega latencia frente a un despliegue monolítico. El objetivo de este trabajo es someter esa premisa a un contraste estadístico formal, usando Python (numpy [1], pandas [3], matplotlib [2], seaborn [6], scipy [5]) sobre la variable *latencia de respuesta HTTP (ms)* agrupada por *tipo de arquitectura* (Monolito vs. Microservicios), y emitir una recomendación técnica de ingeniería respaldada por un nivel de significancia $\alpha = 0{,}05$.

2. Metodología

2.1. Dataset

Conjunto sintético paramétrico justificado ($N = 500$, semilla 42). Las latencias se modelan con distribuciones gamma de cola derecha, típicas de tiempos de servicio de red, más un piso mínimo de procesamiento local:

$$Y_{\text{monolito}} = \Gamma(k = 2{,}5,\ \theta = 35) + 40, \qquad \mathbb{E}[Y_{\text{monolito}}] = 127{,}5 \text{ ms},$$

$$Y_{\text{micro}} = \Gamma(k = 2{,}2,\ \theta = 40) + 45, \qquad \mathbb{E}[Y_{\text{micro}}] = 133 \text{ ms}.$$

El grupo microservicios usa menor forma (k: 2,5 → 2,2) y mayor escala (θ: 35 → 40) para representar mayor varianza por saltos de red y serialización. Los datos crudos y depurados se alojan en `data/` y el cuaderno ejecutable en `notebooks/`.

2.2. Tratamiento de atípicos

Para cada grupo se calcula el rango intercuartílico $\text{IQR} = Q_3 - Q_1$; se eliminan observaciones fuera de $[Q_1 - 1{,}5\,\text{IQR},\ Q_3 + 1{,}5\,\text{IQR}]$. Resultado: 17 atípicos (10 Monolito, 7 Microservicios), $N_{\text{dep}} = 483$.

2.3. Métodos estadísticos

Los criterios de inferencia siguen la práctica estándar de ingeniería [4, 7]:

- Error estándar de la media: $\sigma_{\bar{x}} = s/\sqrt{n}$; verificación empírica con $k = 1000$ remuestreos con reemplazo para $n \in \{5, 15, 30, 60\}$.
- Intervalo de confianza para μ: $\bar{x} \pm t_{\alpha/2,\,n-1} \cdot s/\sqrt{n}$. Se usa t-Student porque σ es desconocida y se sustituye por s; para $n \geq 30$ es asintóticamente equivalente a Z.
- Diferencia de medias: si Levene no rechaza homogeneidad ($p \geq 0{,}05$), varianzas combinadas $s_p^2 = \frac{(n_1-1)s_1^2+(n_2-1)s_2^2}{n_1+n_2-2}$; en caso contrario, corrección de Welch con grados de libertad de Satterthwaite.
- Contraste bilateral de medias mediante `scipy.stats.ttest_ind`, con hipótesis $H_0 : \mu_1 = \mu_2$ frente a $H_1 : \mu_1 \neq \mu_2$.

3. Bloque 1: Estadística descriptiva y limpieza

Tras la depuración, los estadísticos por grupo son:

Cuadro 1: Estadísticos descriptivos de la latencia (ms) por arquitectura ($N_{\text{dep}} = 483$).

Grupo	$\bar{x}$	Mediana	s	s^2	Asimetría	Curtosis
Monolito	123,200	117,012	43,529	1894,787	0,616	$-0,159$
Microservicios	124,614	113,969	48,204	2323,634	0,770	$-0,094$

Fuente: elaboración propia.

Ambas distribuciones presentan asimetría positiva (cola derecha), coherente con colas de latencia de red, y curtosis cercana a cero (colas menos pesadas que la normal). Microservicios muestra mayor dispersión ($s = 48{,}20$ vs. 43,53 ms). En los histogramas con KDE (Figura 1) se aprecia el solapamiento sustancial de ambas poblaciones; los boxplots (Figura 2) confirman medianas y rangos comparables.

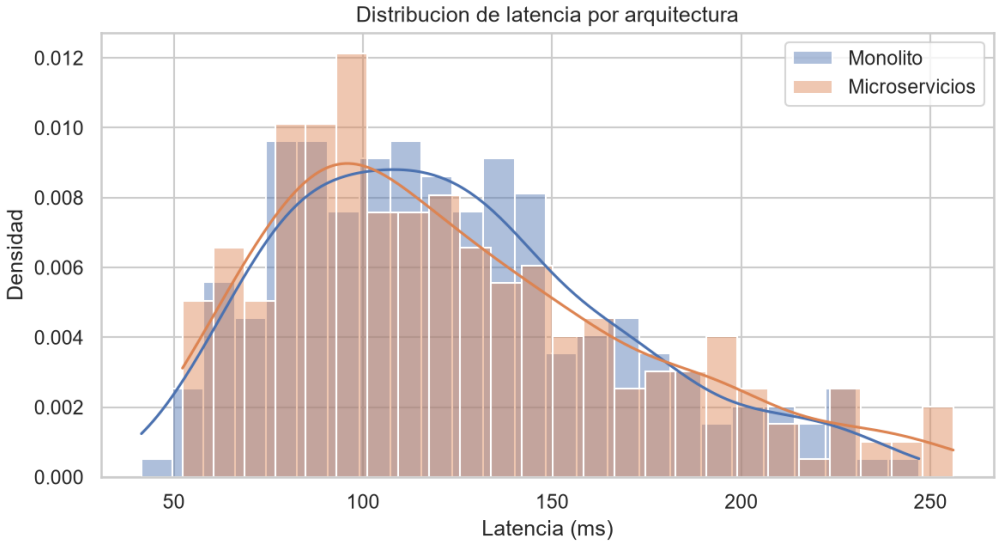


Figura 1: Histogramas con densidad KDE de la latencia por arquitectura.
Fuente: elaboración propia.

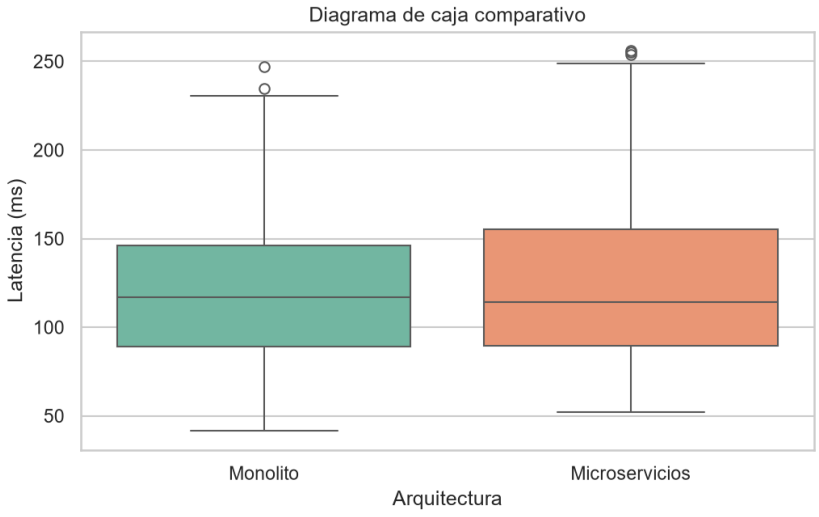


Figura 2: Diagramas de caja comparativos por arquitectura.
Fuente: elaboración propia.

4. Bloque 2: Teorema Central del Límite

Se extrajeron $k = 1000$ medias muestrales con reemplazo para cada $n \in \{5, 15, 30, 60\}$ a partir de la muestra global depurada ($\bar{x} = 123{,}91$ ms, $s = 45{,}90$ ms). La cuadrícula de la Figura 3 muestra cómo las medias se concentran y ganan simetría al aumentar n.

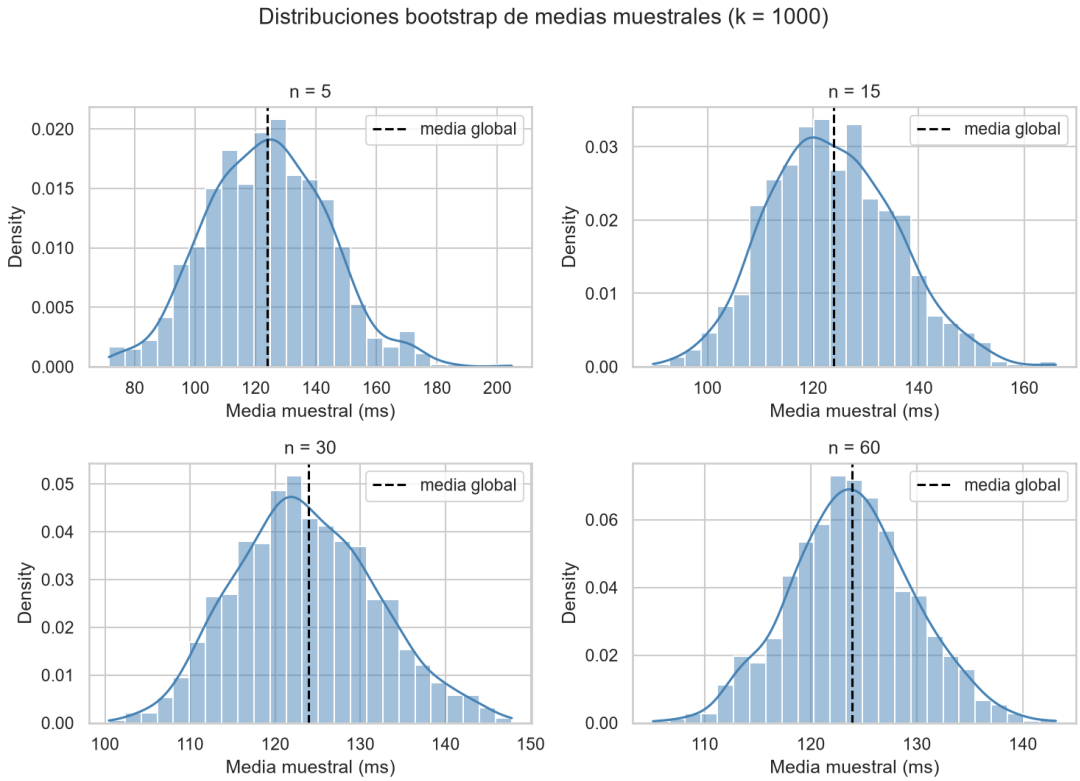


Figura 3: Distribuciones bootstrap de medias muestrales ($k = 1000$).
Fuente: elaboración propia.

Cuadro 2: Error estándar empírico vs. teórico $s/\sqrt{n}$.

n	Media de medias	SE empírico	SE teórico	Diferencia (%)
5	123,918	19,886	20,526	3,12
15	123,615	11,932	11,851	0,68
30	123,658	8,236	8,380	1,72
60	123,882	5,860	5,926	1,11

Fuente: elaboración propia.

El SE empírico difiere del teórico en menos de 3,2 % en todos los casos, validando la forma $\sigma_{\bar{x}} = s/\sqrt{n}$. Las distribuciones muestrales de medias convergen a la normal conforme crece n, evidencia directa del TCL aun partiendo de poblaciones asimétricas.

5. Bloque 3: Intervalos de confianza

5.1. Media general

Usando t-Student con $n = 483$ (σ desconocida):

Cuadro 3: Intervalos de confianza para la media general de latencia (ms).

Confianza	Límite inferior	Límite superior
90 %	120,470	127,353
95 %	119,808	128,015
99 %	118,511	129,312

Fuente: elaboración propia.

5.2. Diferencia de medias

La prueba de Levene obtuvo $W = 1{,}8419$ con $p = 0{,}1754 \geq 0{,}05$: no se rechaza la homogeneidad de varianzas, por lo que se usa el estimador de varianzas combinadas con $n_1 + n_2 - 2 = 481$ grados de libertad:

$$(\bar{x}_1 - \bar{x}_2) \;\pm\; t_{0{,}025,\,481} \cdot s_p \sqrt{\tfrac{1}{n_1} + \tfrac{1}{n_2}} \;=\; -1{,}414 \;\pm\; 8{,}21 \;=\; [-9{,}63,\; 6{,}80] \text{ ms}.$$

El valor cero pertenece al intervalo ($0 \in IC$): a nivel poblacional la diferencia $\mu_{\text{Monolito}} - \mu_{\text{Microservicios}}$ perfectamente puede ser nula. Prácticamente no existe evidencia de que la arquitectura de microservicios agregue latencia media en este experimento; cualquier diferencia observable es compatible con el ruido muestral.

6. Bloque 4: Teoría de decisión

6.1. Planteamiento

$$H_0:\ \mu_{\text{Monolito}} = \mu_{\text{Microservicios}} \qquad \text{vs.} \qquad H_1:\ \mu_{\text{Monolito}} \neq \mu_{\text{Microservicios}}, \quad \alpha = 0{,}05.$$

6.2. Matriz de decisión y costos

6.3. Resultado y veredicto

Con `scipy.stats.ttest_ind` (varianzas combinadas, $n_1 = 240$, $n_2 = 243$):

$$t = -0{,}3384, \qquad p\text{-valor} = 0{,}7352.$$

Como $p = 0{,}7352 \geq \alpha = 0{,}05$, el veredicto es **no rechazar** H_0. No hay evidencia estadística de diferencia en la latencia media entre monolito y microservicios; el IC de la diferencia lo confirma al contener al cero.

6.4. Recomendación técnica

La decisión de arquitectura debe sustentarse en criterios de escalabilidad, despliegue independiente y resiliencia, no en la latencia media observada. Si se migra a microservicios, se recomienda instrumentar telemetría de latencia en producción y dimensionar una tolerancia de hasta $\sim$10 ms de sobrecarga potencial, margen sugerido por el valor absoluto del límite inferior del IC 95 % de la diferencia ($|-9{,}63| \approx 10$ ms); el límite superior (+6,80 ms) indica que microservicios incluso podría ser marginalmente más rápido.

Cuadro 4: Matriz de decisión con costo operativo de cada error.

H verdadera	Decisión	Resultado	Costo / consecuencia técnica
H_0	Rechazar H_0	Error Tipo I (α)	Migrar a microservicios sin beneficio medible: mayor complejidad operativa e infraestructura.
H_0	No rechazar H_0	Correcta ($1-\alpha$)	Se conserva el monolito con evidencia de ausencia de sobrecarga significativa.
H_1	Rechazar H_0	Correcta ($1-\beta$)	Migración justificada: se absorbe la sobrecarga planificando latencia.
H_1	No rechazar H_0	Error Tipo II (β)	No migrar existiendo sobrecarga real: latencia evitable en producción.

Fuente: elaboración propia.

7. Reproducibilidad

Todo el análisis es reproducible de extremo a extremo:

- Entorno verificado: Python 3.14.7, numpy 2.5.4, pandas 3.0.6, scipy 1.18.1, matplotlib 3.11.2, seaborn 0.13.2; resultados idénticos en el kernel de Kaggle (Python 3.13) y en Python 3.10 con numpy 2.2.
- Semilla fija: `np.random.seed(42)` en la primera celda del cuaderno.
- Ejecución local (raíz del repositorio): `jupyter nbconvert -to notebook -execute` sobre `notebooks/examen_fcd_u2.ipynb`.
- Ejecución en la nube: https://www.kaggle.com/code/dlthhb/examen-fcd-u2-monolito-vs-microservicios.
- Entregables en el repositorio: `data/` (CSV original y depurado), `notebooks/` (cuaderno con salidas), `docs/` (este reporte y figuras).

8. Limitaciones

- Con $n_1 = 240$, $n_2 = 243$ y $s_p \approx 45{,}9$ ms, el tamaño muestral detecta con 80 % de potencia diferencias de magnitud $\gtrsim 11{,}7$ ms; diferencias menores quedarían no detectadas.
- No rechazar H_0 no demuestra equivalencia: solo indica ausencia de evidencia de diferencia. Para afirmar equivalencia se requeriría un diseño de hipótesis de equivalencia (TOST) con un margen práctico definido.
- Los datos son sintéticos: los parámetros capturan el comportamiento esperado de latencias de red, pero la validez externa frente a cargas reales de producción no está garantizada.

9. Conclusiones

1. La limpieza con 1,5×IQR eliminó 17 atípicos (3,4 %); los estadísticos descriptivos revelan asimetría positiva moderada en ambos grupos.

2. La simulación del TCL ($k = 1000$) verificó que el SE empírico coincide con $s/\sqrt{n}$ (diferencias $< 3{,}2\,\%$) y que las medias muestrales convergen a la normal.

3. El IC 95 % para $\mu_{\text{Monolito}} - \mu_{\text{Microservicios}}$ es $[-9{,}63,\ 6{,}80]$ ms: contiene al cero.

4. Con $\alpha = 0{,}05$, la prueba t ($p = 0{,}74$) no rechaza H_0. Técnicamente, la sobrecarga de red de microservicios no demuestra un incremento significativo de latencia en este escenario.

Referencias

[1] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[2] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[3] Wes McKinney. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[4] Douglas C. Montgomery and George C. Runger. *Probabilidad y Estadística Aplicada a la Ingeniería*. Limusa Wiley, 4 edition, 2021.

[5] Pauli Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[6] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021.

[7] Larry Wasserman. *All of Statistics: A Concise Course in Statistical Inference*. Springer, 2004.